

Snacki

Jus & délices — Nouadhibou, Mauritanie

LIVRABLE 1 · ÉTAPE 1 SUR 6

L'application client

Menu, panier et commande WhatsApp

Ce qui a été construit, comment ça marche, pourquoi ces choix, et ce qu'il reste à valider.

تطبيق الزبائن: القائمة، السلة، والطلب عبر واتساب

Version **v1.1 — 25 septembre 2026**

Périmètre **Côté client uniquement (la caisse arrive à l'étape 2)**

Statut **Livrée · visuels à valider**

Sources **Menu Snacki, Suivi Dashboard 15.xlsx, Rapport d'analyse globale, Architecture & Sécurité v2**

1

App client

2

Caisse

3

Base de données

4

Achats & stock

5

Tableau de bord

6

Croissance

Synthèse

L'étape 1 donne aux clients de Snacki un moyen simple de commander depuis leur téléphone : ils choisissent dans le menu, remplissent trois informations, et la commande part sur le WhatsApp du snack, déjà rédigée et chiffrée.

9

produits du menu, avec prix en MRU

3

écrans pour commander : menu, panier, infos

2

langues : français et arabe (droite à gauche)

0

compte, installation ou serveur nécessaires

CE QUI EST FAIT

- Menu réel (prix, noms arabes du menu imprimé, descriptions courtes validées).
- Panier avec compteurs, total en direct, panier gardé si le client ferme la page.
- Formulaire court avec contrôles : prénom, téléphone mauritanien, retrait ou livraison, paiement.
- Message WhatsApp structuré avec numéro de commande.
- Tests sur téléphone, en français, en arabe et en mode sombre.

CE QUI RESTE À VALIDER

- **Visuels produits** : les photos d'origine (floues, mal cadrées) sont remplacées par 9 illustrations dessinées dans le style de la marque (section 4).
- Deux descriptions à confirmer : jus fraise « au lait », mojito « sans alcool ».

Sommaire

1	Le projet en 6 étapes	2
2	Objectif de l'étape 1 et critères de réussite	3
3	Le parcours du client, écran par écran	4
4	Le menu dans l'application	6
5	Comment l'application est construite	7
6	Français et arabe : le sens de lecture	8
7	La commande WhatsApp	10
8	Sécurité : menaces et protections	11
9	Tests réalisés et bugs corrigés	13
10	Décisions, points ouverts et limites	15
11	Étape suivante : la caisse	16
A	Glossaire	17

1 Le projet en 6 étapes

L'application Snacki doit servir deux publics très différents : des **clients** qui veulent commander vite, sans compte ni explication, et des **gestionnaires** qui ont besoin d'outils fiables pour tenir la caisse, les achats et les chiffres. On la construit par étapes, chacune utilisable seule et validée avant de passer à la suivante.



#	Ce que l'étape remplace aujourd'hui	Pourquoi à ce moment-là
1	Le menu imprimé et les messages WhatsApp écrits à la main par les clients	Visible tout de suite par les clients, sans rien changer à l'organisation du snack.
2	L'onglet « Ventes » de l'Excel, saisi en texte libre (« 1 crêpe + 2 jus fraise... »)	Tant que les ventes sont du texte libre, on ne peut pas compter les produits vendus.
3	L'architecture v2 sans serveur (aucune donnée conservée, code PIN visible dans le code)	Nécessaire pour que les commandes client arrivent dans la caisse et pour sécuriser l'accès gestionnaire.
4	L'onglet « Les achats matières premières »	S'appuie sur la base de données de l'étape 3.
5	Le rapport d'analyse fait à la main (PDF)	Ne vaut que si les données des étapes 2 à 4 sont propres.
6	Recommandations R1 (fidélité) et R7 (commande en ligne) du rapport	Une fois l'outil interne solide.

À APPRENDRE

Cette façon de faire s'appelle le **développement incrémental** : on livre une petite partie utile, on la fait valider, puis on construit la suivante sur une base déjà éprouvée. L'ordre des étapes suit les **dépendances** : le tableau de bord (5) dépend de données propres, qui dépendent de la caisse (2) et de la base de données (3).

2 Objectif de l'étape 1 et critères de réussite

Objectif : qu'un client de Nouadhibou puisse passer une commande complète et sans ambiguïté en moins d'une minute, depuis son téléphone, en français ou en arabe.

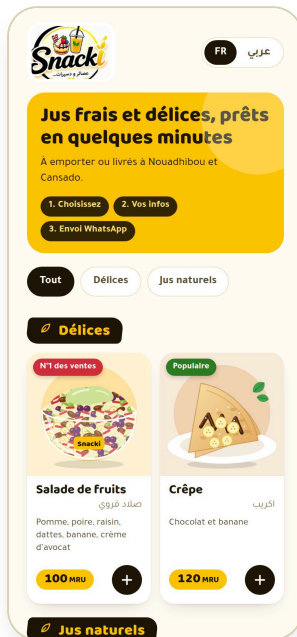
Critère	Comment on le vérifie	Résultat
Commander en 3 écrans maximum	Menu → panier → infos → message prêt	Atteint
Aucun compte, aucune installation	Le client ouvre un lien, c'est tout	Atteint
Bilingue, arabe lu de droite à gauche	Bouton FR / عربي, mise en page inversée	Atteint
Commande lisible sans relecture par le snack	Message structuré : lignes, total, paiement, contact	Atteint
Informations fiables	Téléphone et repère de livraison contrôlés avant envoi	Atteint
Visuels fidèles et appétissants	Validation par l'équipe	À valider

3 Le parcours du client, écran par écran

Le client ne voit jamais plus d'une décision à la fois. La barre du bas lui rappelle en permanence le nombre d'articles et le total.

À RETENIR

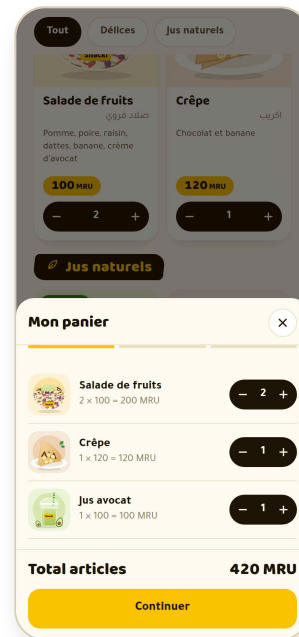
La commande n'est **pas enregistrée par l'application** à cette étape : c'est le message WhatsApp qui fait foi. Le snack confirme ensuite la commande et le délai au client dans la conversation. C'est le fonctionnement actuel, en plus propre ; l'étape 3 fera arriver les commandes directement dans la caisse.



① **Accueil.** Le logo, une phrase qui dit ce qu'on peut faire, les 3 étapes, puis le menu classé en *Délices* et *Jus naturels*.



② **Choix.** Le bouton + devient un compteur. La barre noire apparaît dès le premier article, avec le total en MRU.



③ **Panier.** Chaque ligne montre le calcul ($2 \times 100 = 200$ MRU). On peut encore ajuster les quantités.

④ **Contrôles.** Si un champ manque, le message dit quoi corriger, et le curseur se place sur le premier champ en erreur.

⑤ **Infos.** En livraison, le quartier et un repère sont demandés. Les frais sont confirmés sur WhatsApp.

⑥ **Envoi.** Numéro de commande, message prêt, bouton WhatsApp, et un bouton « Copier » si WhatsApp ne s'ouvre pas.

4 Le menu dans l'application

Les noms arabes reprennent ceux du menu imprimé. Les prix sont affichés en **MRU** (décision du 25/09) : le menu imprimé indique 1 000 et 1 200, qui correspondent à 100 et 120 MRU.

Produit	Nom arabe	Catégorie	Prix	Description affichée	Badge
Salade de fruits	صلاّد فروي	Délices	100 MRU	Pomme, poire, raisin, dattes, banane, crème d'avocat	N°1 des ventes
Crêpe	اكريب	Délices	120 MRU	Chocolat et banane	Populaire
Jus avocat	عصير أفوكا	Jus	100 MRU	Avocat mixé au lait	Populaire
Jus fraise	عصير الفراولة	Jus	100 MRU	Fraises mixées au lait à confirmer	
Jus mangue	عصير مانكو	Jus	100 MRU	Mangue bien fraîche	
Jus orange	عصير البرتقال	Jus	100 MRU	Oranges pressées	
Jus cocktail	عصير كوكتيل	Jus	100 MRU	Mélange de fruits	
Jus banane-fraise	عصير بنانة والفراولة	Jus	100 MRU	Banane et fraise	
Mojito	موخيتو	Jus	100 MRU	Menthe et citron vert, sans alcool à confirmer	

Les visuels produits



Les photos prises au snack étaient floues, cadrées de travers et prises à la main, avec des filigranes TikTok. Elles sont remplacées par des **illustrations dessinées par le code** (format SVG) : chaque produit est servi dans le vrai gobelet Snacki à couvercle dôme, avec l'étiquette jaune, la couleur de sa boisson et ses fruits posés devant. Avantages : un style homogène et net sur tous les écrans, aucun droit d'image à gérer, et 300 Ko d'images en moins. Limite : ce ne sont pas des photos réelles. Si l'équipe préfère de vraies photos, l'app accepte une photo par produit (champ **photo**) qui prend alors la place de l'illustration.

D'où viennent les badges

Les badges ne sont pas décoratifs : ils reprennent le classement des ventes J1 à J21 du rapport d'analyse (salade de fruits en tête, puis crêpe et jus d'avocat). Mettre en avant ce qui se vend le mieux aide le client qui hésite et augmente le panier moyen. Quand la caisse (étape 2) comptera les ventes réelles, ces badges pourront être calculés automatiquement.

Produits non repris

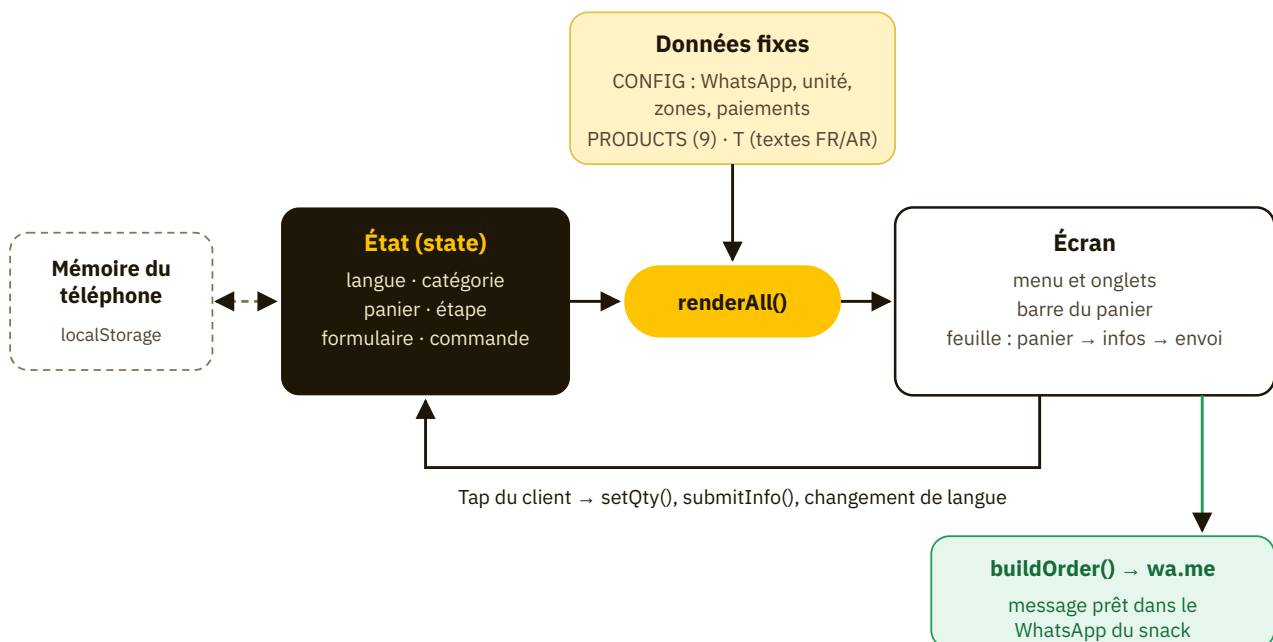
Les commandes de l'Excel mentionnent aussi des desserts (tiramisu, crème) et des mélanges (avocat-banane, mangue-banane). Ils ne sont pas sur le menu imprimé, donc pas dans l'app. Les ajouter demande seulement une ligne par produit dans la liste **PRODUCTS** (section 5).

À APPRENDRE : MRU ET MRO

Depuis 2018, l'ouguiya « nouvelle » (MRU) vaut 10 anciennes ouguiyas (MRO). Beaucoup de prix se disent encore en ancienne ouguiya : c'est le cas du menu imprimé et de l'Excel de suivi (loyer 90 000, jus 1 000). Le rapport d'analyse les étiquette en MRU : il faudra diviser ces montants par 10 lors du tableau de bord (étape 5).

5 Comment l'application est construite

L'application tient dans **un seul fichier** `index.html` (839 lignes, 52 Ko) plus le logo (49 Ko) : les visuels produits sont dessinés par le code (section 4). Elle n'utilise aucune bibliothèque externe : du HTML, du CSS et du JavaScript « pur ». C'est ce qu'on appelle une **application monopage** (SPA) : la page ne se recharge jamais, c'est le code qui redessine ce qui change.



Le principe : l'état, puis le rendu

Tout ce qui peut changer est rangé dans un seul objet, `state`. Quand le client touche un bouton, le code **modifie l'état**, puis appelle `renderAll()` qui **redessine l'écran à partir de l'état**. L'écran n'est jamais modifié « à la main » : il est toujours la traduction de l'état. C'est l'idée centrale des outils modernes comme React ou Vue, appliquée ici sans bibliothèque.

```
const state = {
  lang: store.get('lang', 'fr'), // langue choisie, retenue d'une visite à l'autre
  cat: 'all', // onglet : tout, délices ou jus
  cart: sanitizeCart(store.get('cart', {})), // { salade: 2, avocat: 1 }
  step: null, // null | 'cart' | 'info' | 'done'
  form: { name, phone, mode, zone, landmark, pay, note },
  order: null // { no: 'SNK-0925-596', text: '...' }
};
```

Le panier ne stocke que des **identifiants et des quantités**, jamais des prix. Le prix est toujours relu dans la liste `PRODUCTS` au moment du calcul : si un prix change, aucun panier ancien ne peut garder l'ancien.

Les trois blocs de données fixes

Bloc	Contenu	Qui le modifie
CONFIG	Numéro WhatsApp, unité (MRU), quartiers de livraison, moyens de paiement	Une seule fois, en cas de changement du snack
PRODUCTS	Les 9 produits : identifiant, catégorie, prix, photo, noms et descriptions FR/AR, badge	À chaque changement de carte ou de prix
T	Tous les textes de l'interface, en français et en arabe	Pour reformuler un bouton ou un message

À RETENIR

Séparer **données**, **état** et **rendu** permet de changer un prix ou une traduction sans toucher à la logique. À l'étape 3, **PRODUCTS** ne sera plus écrit dans le fichier : il viendra de la base de données, et la gestionnaire pourra changer un prix ou masquer un produit en rupture depuis la caisse.

6 Français et arabe : le sens de lecture

Un bouton en haut de l'écran bascule toute l'application. En arabe, la page entière passe **de droite à gauche** : le logo, les onglets, les badges, les flèches et les boutons changent de côté.



Menu en arabe. Le nom français reste en petit sous le nom arabe, pour les clients qui lisent les deux.



Confirmation en arabe. Le message WhatsApp est traduit, avec le même numéro de commande.

Comment ça marche

1. Le code met l'attribut `dir="rtl"` sur la page. Le navigateur inverse alors automatiquement l'ordre des éléments.
2. Le CSS utilise des propriétés **logiques** : `inset-inline-start` (« au début de la ligne ») au lieu de `left`. En arabe, le début de la ligne est à droite, donc le badge « الأكثر طلبًا » passe à droite tout seul.

3. Les icônes qui indiquent une direction (flèche retour) sont retournées en miroir ; les icônes neutres (+, -, ×) ne bougent pas.

Le piège des nombres

Pendant les tests, le prix « 1 000 » s'affichait « 000 1 » en arabe. Le navigateur voyait deux nombres séparés par une espace, et les rangeait de droite à gauche comme des mots arabes. La correction entoure chaque nombre de deux caractères invisibles, U+2066 et U+2069, qui disent au navigateur : « ce morceau se lit de gauche à droite, quoi qu'il arrive autour ».

```
const fmt = n => '' + fmtPlain(n).replace(/ /g, ' ') + '';  
// U+2066 = « début d'un îlot gauche-à-droite », U+2069 = « fin de l'îlot »
```

Dans le message WhatsApp, ces caractères sont remplacés par des marques U+200E (LRM), plus anciennes et reconnues par tous les téléphones : certains anciens modèles affichent les isolats comme des carrés.

À APPRENDRE : L'ALGORITHME BIDIRECTIONNEL

Unicode définit comment afficher un texte qui mélange des écritures de sens opposés (*bidi*). Les lettres arabes sont « fortes » droite-à-gauche, les lettres latines « fortes » gauche-à-droite, et les chiffres et espaces « faibles » : ils prennent la direction de ce qui les entoure. D'où le bug. Les « isolats » (U+2066...U+2069) ou la balise HTML `<bdi>` protègent un morceau de texte de son voisinage.

7 La commande WhatsApp

À l'étape 1, WhatsApp sert de canal de commande : c'est déjà l'outil du snack et de ses clients, il ne coûte rien, et le snack garde une trace écrite de chaque commande. L'app ne fait que **rédiger le message à la place du client**, sans oubli ni ambiguïté.

Message généré (français)

Nouvelle commande Snacki
N° : SNK-0925-596

```
2 x Salade de fruits - 200
1 x Crêpe - 120
1 x Jus avocat - 100
```

Total : 420 MRU + livraison à confirmer
 Paiement : Bankily

Nom : Aicha
Tél : +222 37 93 94 09
Mode : Livraison – Cansado
Repère : Près de la mosquée, maison bleue

Même commande (arabe)

طلب جديد من Snacki

رقم : SNK-0925-596

200 – 2 × صلابد فروي
120 – 1 × اڪريپ
100 – 1 × عصير أفوكا

*المجموع : 420 MRU + التوصيل يُؤكّد لاحقًا
الدفع : بنكيلى

الاسم : Aïcha
 الهاتف : +222 37 93 94 09
 الاستلام : توصيل – كاصادو
 العنوان : Près de la mosquée, maison bleue

Les astérisques mettent le texte en gras dans WhatsApp. Le téléphone est reformaté par paires de chiffres pour être lisible par l'équipe. Dans la version arabe, les nombres sont encadrés de marques invisibles LRM (U+200E) pour garder le bon ordre des chiffres (section 9).

Du message au lien WhatsApp

WhatsApp accepte des liens de la forme `https://wa.me/<numéro>?text=<message>`. Le numéro est écrit au format international sans « + » (222 pour la Mauritanie, puis 37939409). Le message doit être **encodé** : les espaces deviennent `%20`, les retours à la ligne `%0A`, « é » devient `%C3%A9`. La fonction JavaScript `encodeURIComponent()` s'en charge. Début du lien réel :

https://wa.me/22237939409?text=*Nouvelle%20commande%20Snacki*%0A%C2%B0%203A%20SNK-0925-596%0A%0A2%20C3%97%20Salade%20de%20fruits%20E2%80%94%2000%0A1%20C3%97%20Cr%C3%Aape%20E2%80%94%20120...

Le numéro de commande

Format **SNK-MMJJ-XXX** : le mois et le jour, puis 3 chiffres tirés au hasard (100 à 999) avec le générateur cryptographique du navigateur. Il remplace la numérotation « cmd1, cmd2... » de l'Excel, qui repart à zéro chaque jour et ne permet pas de retrouver une commande.

LIMITE CONNUE : DEUX COMMANDES PEUVENT AVOIR LE MÊME NUMÉRO

Chaque téléphone tire son numéro seul, sans savoir ce que les autres ont tiré. Avec 900 numéros possibles par jour, la probabilité qu'au moins deux commandes du même jour partagent un numéro (le « paradoxe des anniversaires ») est d'environ **4 % un jour à 9 commandes** (moyenne J1–J21) et **11 % un jour à 15 commandes**. Sur un mois, un doublon est donc probable. Ce risque grimpe vite : il suffit de 36 commandes le même jour pour dépasser 50 %, la même mathématique qui rend les collisions de hachage dangereuses en cryptographie. C'est acceptable tant que l'équipe lit aussi le nom et le téléphone ; l'étape 3 attribuera des numéros uniques côté serveur (1, 2, 3... sans trou ni doublon).

8 Sécurité : menaces et protections

À l'étape 1, l'application ne contient **aucun secret** (pas de code PIN, pas de clé, pas d'accès gestionnaire) et **aucun serveur** : tout se passe dans le téléphone du client, puis dans WhatsApp. La surface d'attaque est donc petite. Elle n'est pas nulle : le client tape du texte, et ce texte est réaffiché à l'écran puis envoyé au snack.

Menace	Scénario concret	Protection mise en place
Injection de code (XSS) OWASP A03	Un prénom comme <code></code> serait exécuté comme du code s'il était inséré tel quel dans la page.	Les textes saisis sont toujours insérés comme du texte (<code>textContent</code> , <code>createTextNode</code>), jamais comme du HTML. Testé : le prénom piégé s'affiche tel quel, rien ne s'exécute.
Données locales falsifiées OWASP A08	La mémoire du téléphone (<code>localStorage</code>) est modifiable par l'utilisateur : il peut y écrire un panier de 1 milliard de salades ou un produit inventé.	Au chargement, <code>sanitizeCart()</code> ne garde que les produits existants et les quantités entières de 1 à 99. Testé avec un panier piégé : seules les lignes valides sont gardées.
Faux message dans le message	Un client écrit dans « Remarque » un retour à la ligne suivi de « Total : 0 MRU », pour faire croire à un autre total.	Les retours à la ligne sont supprimés de chaque champ libre (fonction <code>clean</code>) : la remarque reste sur sa ligne, préfixée de « Remarque : ». Le vrai total est toujours en gras, calculé par l'app.
Informations inutilisables OWASP A04	Numéro faux ou incomplet, livraison sans adresse : la commande ne peut pas être honorée.	Prénom ≥ 2 lettres ; téléphone : exactement 8 chiffres commençant par 2, 3 ou 4 (préfixes des opérateurs mauritaniens) ; repère obligatoire en livraison ; longueurs maximales sur chaque champ.
Numéro de commande prévisible	Un numéro calculable à l'avance faciliterait une fausse réclamation.	Tirage par <code>crypto.getRandomValues()</code> (générateur cryptographique) plutôt que <code>Math.random()</code> .
Stockage indisponible	Navigation privée ou stockage bloqué : l'app pourrait planter.	Chaque lecture et écriture du stockage est protégée (<code>try/catch</code>) ; l'app fonctionne sans, seul le panier n'est plus retenu.

Pourquoi `textContent` plutôt qu' `innerHTML`

```
// Dangereux : le navigateur interprète le texte comme du HTML
el.innerHTML = 'Nom : ' + prenom; // <img onerror=...> s'exécute

// Sûr : le navigateur affiche le texte tel quel
el.textContent = 'Nom : ' + prenom; // affiche littéralement <img onerror=...>
```

Toute l'interface est construite par une petite fonction `h(balise, attributs, enfants)` qui crée les éléments un par un et insère les textes comme des nœuds de texte. Les seuls `innerHTML` du fichier dessinent les icônes et les illustrations des produits, à partir de chaînes écrites dans le code, jamais de ce que tape le client.

Ce que l'étape 1 ne protège pas, et quand ce sera traité

Risque résiduel	Pourquoi	Traitement prévu
Fausse commande envoyée directement sur WhatsApp	N'importe qui peut écrire au snack un message qui ressemble à une commande de l'app.	Aujourd'hui : l'équipe confirme chaque commande dans la conversation. Étape 3 : commandes enregistrées côté serveur, seul le serveur calcule les totaux.
Prix modifiés sur le téléphone du client	Le code tourne chez le client, qui peut le modifier.	L'équipe vérifie le total avec le menu. Étape 3 : le serveur recalcule chaque total.
Coordonnées retenues sur un téléphone partagé	Le prénom, le téléphone et le repère sont gardés pour la prochaine commande.	Données peu sensibles ; ajouter un bouton « Effacer mes infos » à l'étape 6.
Doublons de numéros de commande	Tirage au hasard sur chaque téléphone (section 7).	Étape 3 : numérotation côté serveur.

LIEN AVEC LE DOCUMENT « ARCHITECTURE & SÉCURITÉ V2 »

Ce document signalait une faiblesse majeure de la v2 : l'empreinte SHA-256 du code PIN gestionnaire est lisible dans le code public, et un PIN à 4 chiffres (10 000 possibilités) se retrouve en quelques millisecondes à partir de cette empreinte. L'étape 1 **ne contient plus aucun accès gestionnaire** : ce risque disparaît de l'app client. L'accès gestionnaire reviendra à l'étape 3, **vérifié par un serveur**, comme le recommandait la feuille de route sécurité (priorité P1).

À APPRENDRE : NE JAMAIS FAIRE CONFIANCE AU CLIENT

Tout ce qui s'exécute dans le téléphone du client (code, stockage, calculs) peut être lu et modifié par lui. Une règle de base en sécurité web : le client sert à **afficher et saisir**, le serveur à **décider** (prix, totaux, droits d'accès). L'étape 1 n'a pas de serveur, donc la décision finale revient à l'humain qui lit le message ; l'étape 3 la confiera au serveur.

9 Tests réalisés et bugs corrigés

Les tests ont été automatisés avec **Playwright**, un outil qui pilote un vrai navigateur (Chromium) comme le ferait un client : il touche les boutons, remplit les champs, lit ce qui s’affiche et prend des captures. Écran simulé : 390 × 844 px (taille d’un téléphone courant).

N°	Test	Résultat attendu	Résultat
T01	Ajouter un produit	Le + devient compteur, la barre du panier apparaît	OK
T02	Totaux : 2 salades + 1 crêpe + 1 jus avocat	$2 \times 100 + 120 + 100 = 420$ MRU partout	OK
T03	Valider un formulaire vide	Messages d’erreur, curseur sur le premier champ	OK
T04	Téléphones : 37939409, 22123456, 41445574, « 37 93 94 09 »	Acceptés	OK
T05	Téléphones : 12345678, 5123456, 3793940, 379394099, abcdefgh	Refusés	OK
T06	Choisir « Livraison »	Quartier et repère apparaissent, repère obligatoire	OK
T07	Contenu du message WhatsApp	Lignes, total, paiement, nom, téléphone, mode, repère	OK
T08	Passer en arabe sur l’écran final	Message traduit, même numéro de commande	OK
T09	Recharger la page avec un panier	Panier retrouvé	OK
T10	Panier piégé dans le stockage (quantité 1 milliard, produit inventé, quantité négative)	Lignes invalides ignorées	OK
T11	Prénom contenant du code HTML	Affiché comme texte, aucune exécution	OK
T12	Remarque contenant un retour à la ligne et un faux total	Reste sur une seule ligne « Remarque : ... »	OK
T13	Mode sombre du téléphone	Couleurs adaptées, texte lisible	OK
T14	Largeur de 390 à 400 px	Pas de défilement horizontal	OK
T15	Erreurs JavaScript pendant tout le parcours	Aucune	OK
T16	Message arabe avec un total de 1 120 MRU et un téléphone	« 1 120 » et « +222 37 93 94 09 » dans le bon ordre	OK

Bugs trouvés pendant les tests, puis corrigés

Bug	Cause	Correction
Les prix s’affichaient « 000 1 » en arabe	Algorithme bidirectionnel (section 6)	Isolats Unicode autour des nombres

Bug	Cause	Correction
Dans le message WhatsApp arabe, le téléphone s'affichait « 09 94 93 37 222+ » (trouvé en rédigeant ce livrable)	Même cause : blocs de chiffres séparés par des espaces dans une ligne arabe	Marques LRM (U+200E) autour du téléphone, des montants et du numéro de commande ; reconnues par tous les téléphones, contrairement aux isolats
La feuille du panier, censée être cachée, bloquait les boutons du menu	Une règle CSS <code>display: flex</code> l'emportait sur l'attribut <code>hidden</code>	Règle <code>[hidden]{display:none!important}</code>
Le compteur de quantité débordait de la carte produit	Prix + compteur plus larges que la carte (≈ 146 px utiles)	Le compteur passe sur sa propre ligne, pleine largeur
La notification « ajouté au panier » cachait le bouton de validation	Notification placée en bas de l'écran	Notification déplacée en haut



T13. Mode sombre, en arabe, avec des articles au panier.

10 Décisions, points ouverts et limites

Décisions validées le 25 septembre 2026

Sujet	Décision	Effet dans l'app
Unité affichée	MRU	Jus et salade 100 MRU, crêpe 120 MRU
Frais de livraison	Confirmés sur WhatsApp	Mention « + livraison à confirmer », pas de tarif affiché
Descriptions produits	Validées	Une ligne sous chaque produit, en français et en arabe
Caisse (étape 2)	Sur téléphone, par la gestionnaire ; fiable à un caissier plus tard	Prévoir deux rôles : gestionnaire et caissier aux droits réduits

Points ouverts

Point	Ce qu'il faut	Qui
Visuels des 9 produits	Valider les illustrations. Si l'équipe veut de vraies photos : séance photo au snack (fond uni, lumière du jour, même angle, sans main ni filigrane) ou photos libres de droits envoyées dans la conversation.	Équipe Snacki
Deux descriptions	Confirmer « jus fraise au lait » et « mojito sans alcool »	Équipe Snacki
Mise en ligne pour les clients	L'app est aujourd'hui une page privée de démonstration. La rendre publique demande un hébergement et, idéalement, un nom de domaine (prévu à l'étape 6, avançable si besoin).	À décider

Limites connues de l'étape 1

- **Pas d'enregistrement des commandes** : elles n'existent que dans WhatsApp. On ne peut pas encore compter les ventes automatiquement (étape 2 et 3).
- **Pas de gestion de disponibilité** : un produit en rupture reste commandable ; l'équipe le signale dans la conversation (étape 3).
- **Numéros de commande non garantis uniques** (section 7).
- **Pas de suivi de commande** côté client (étape 6).
- Le bouton WhatsApp peut ne pas s'ouvrir dans certaines prévisualisations ; le bouton « Copier le message » et le numéro affiché servent de solution de secours.

11 Étape suivante : la caisse

Une fois l'étape 1 validée, l'étape 2 donne à la gestionnaire une caisse sur son téléphone. Elle remplace la saisie en texte libre de l'onglet « Ventes » par des commandes **structurées**, que l'on pourra compter, additionner et analyser.

Ce qu'elle permettra

- Saisir une commande au comptoir en quelques taps, avec les mêmes produits et prix que l'app client.
- Reporter une commande reçue sur WhatsApp en la collant ou en la saisissant.
- Suivre les commandes du jour : *reçue* → *en préparation* → *prête* → *livrée*.
- Voir le total du jour et le nombre de commandes, en direct.

Le modèle de données qui arrive

C'est le point d'apprentissage clé de l'étape 2 : passer d'une ligne de texte (« 3 crêpes + 2 salades de fruits ») à des données exploitables.

```
// Une commande
Commande {
  numero      // SNK-0925-596
  date_heure
  canal       // comptoir | whatsapp
  statut      // reçue | en_prépa | ...
  paiement    // espèces | bankily | ...
  client      // prénom, téléphone
  total       // calculé depuis les lignes
}

// Une ligne de commande
LigneCommande {
  commande    // → Commande
  produit     // → Produit (salade...)
  quantite    // 3
  prix_unit   // 120, figé au moment
              // de la vente
}
```

À APPRENDRE : POURQUOI FIGER LE PRIX DANS LA LIGNE

Si le prix de la crêpe passe de 120 à 130 MRU en novembre, les ventes de septembre doivent rester à 120. On copie donc le prix dans chaque ligne au moment de la vente, au lieu de le relire dans la fiche produit. C'est une règle classique de conception de bases de données pour les données **historiques**.

Questions à trancher pour l'étape 2

1. Les commandes WhatsApp doivent-elles apparaître dans la caisse dès l'étape 2 (saisie manuelle), ou attendre l'étape 3 (arrivée automatique) ?
2. Qui peut annuler une commande ou modifier un prix : la gestionnaire seulement, ou aussi le caissier ?
3. Faut-il un ticket (numéro affiché ou imprimé) pour les commandes au comptoir ?

Glossaire

Application monopage (SPA)	Application web qui ne recharge jamais la page : le code JavaScript redessine seulement ce qui change.
PWA	Application web installable sur l'écran d'accueil du téléphone, qui peut fonctionner hors connexion. Prévue à l'étape 6.
État (state)	L'ensemble des informations qui peuvent changer pendant l'utilisation (langue, panier, étape...). L'écran en est la traduction.
Rendu	Le fait de redessiner l'écran à partir de l'état (<code>renderAll()</code>).
localStorage	Petite mémoire du navigateur, propre à chaque téléphone. Modifiable par l'utilisateur, donc jamais digne de confiance.
RTL / LTR	<i>Right-to-left</i> (arabe) et <i>left-to-right</i> (français) : le sens d'écriture.
Bidi	Algorithme Unicode qui décide de l'ordre d'affichage d'un texte mêlant les deux sens.
Propriété CSS logique	Propriété exprimée en « début » et « fin » de ligne plutôt qu'en gauche et droite, qui s'adapte seule au sens de lecture.
Lien profond (wa.me)	Lien qui ouvre directement une application (ici WhatsApp) sur une conversation, avec un message prérempli.
Encodage d'URL	Transformation des caractères spéciaux d'un lien (espaces, accents, retours à la ligne) en codes <code>%XX</code> .
XSS	<i>Cross-site scripting</i> : attaque qui consiste à faire exécuter du code par la page en le glissant dans un texte saisi.
OWASP Top 10	Liste de référence des 10 grandes familles de failles des applications web.
Validation d'entrée	Vérification des champs saisis avant de les utiliser (format du téléphone, longueur, champs obligatoires).
Générateur cryptographique	<code>crypto.getRandomValues()</code> : tirage aléatoire imprévisible, contrairement à <code>Math.random()</code> .
Paradoxe des anniversaires	Phénomène mathématique : des doublons apparaissent bien plus tôt qu'on ne le pense quand on tire au hasard dans un ensemble limité.
SVG	Format d'image décrit par du code (formes, couleurs) plutôt que par des pixels : net à toutes les tailles et très léger.
Playwright	Outil qui pilote un vrai navigateur pour tester une application automatiquement.
MRU / MRO	Ouguiya nouvelle (depuis 2018) et ancienne ouguiya. 1 MRU = 10 MRO.

Document rédigé à partir du menu Snacki, du fichier Suivi Dashboard 15.xlsx, du rapport d'analyse globale (septembre 2026) et du document Architecture & Sécurité v2. Captures réalisées sur la version v1.1 de l'application, le 25 septembre 2026.